Practical11

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment(void *arg)
{
   for (int i = 0; i < ITERATIONS; i++)
   {
      counter++;
   }

   return NULL;
}

int main()
{
   pthread_t threads[NUM_THREADS];

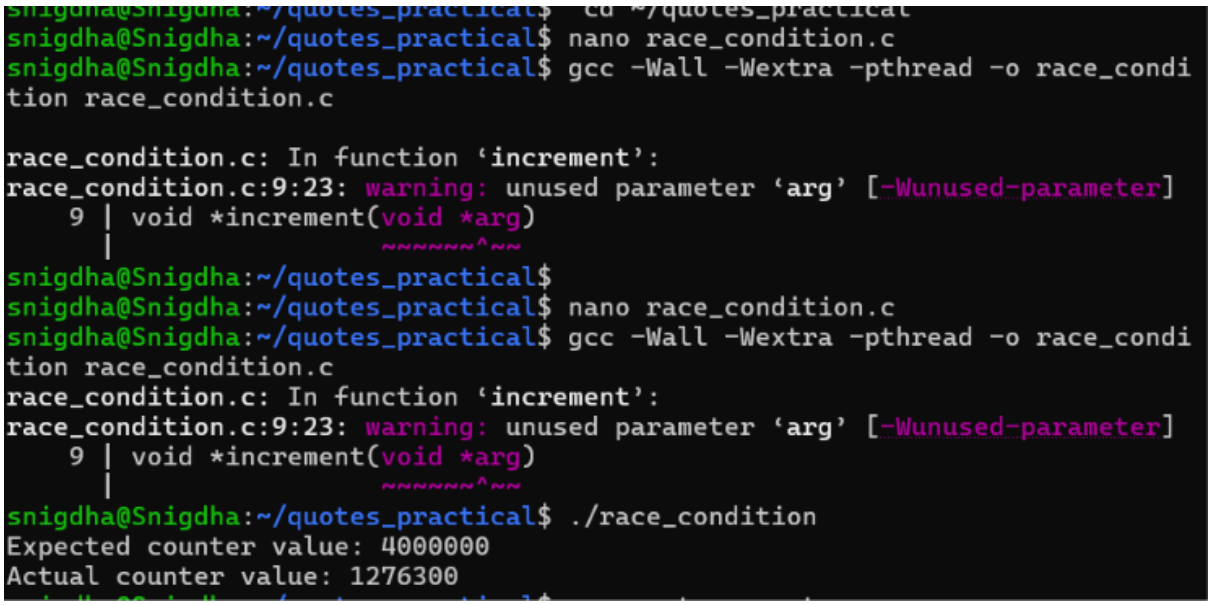
   for (int i = 0; i < NUM_THREADS; i++)
   {
      pthread_create(&threads[i], NULL, increment, NULL);
   }
```

```
    for (int i = 0; i < NUM_THREADS; i++)
    {
        pthread_join(threads[i], NULL);
    }

    printf("Expected counter value: %d\n",
           NUM_THREADS * ITERATIONS);

    printf("Actual counter value: %d\n", counter);

    return 0;
}
```




```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;
```

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;

void *increment(void *arg)
{
   for (int i = 0; i < ITERATIONS; i++)
   {
      pthread_mutex_lock(&lock);

      counter++;

      pthread_mutex_unlock(&lock);
   }

   return NULL;
}

int main()
{
   pthread_t threads[NUM_THREADS];

   for (int i = 0; i < NUM_THREADS; i++)
   {
      pthread_create(&threads[i], NULL, increment, NULL);
   }

   for (int i = 0; i < NUM_THREADS; i++)
   {
```

```
    pthread_join(threads[i], NULL);
  }

  printf("Expected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

  printf("Actual counter value: %d\n", counter);

  pthread_mutex_destroy(&lock);

  return 0;
}
```

```
snigdha@Snigdha:~/quotes_practical$ nano mutex_counter.c
snigdha@Snigdha:~/quotes_practical$ gcc -Wall -Wextra -pthread -o mutex_coun
ter mutex_counter.c
mutex_counter.c: In function ‘increment’:
mutex_counter.c:11:23: warning: unused parameter ‘arg’ [-Wunused-parameter]
   11 | void *increment(void *arg)
      |                 ~~~~~~^~~
snigdha@Snigdha:~/quotes_practical$ ./mutex_counter
Expected counter value: 4000000
Actual counter value: 4000000
snigdha@Snigdha:~/quotes_practical$ ./mutex_counter
Expected counter value: 4000000
Actual counter value: 4000000
```